

TUNISIAN REPUBLIC

MINISTRY OF
HIGHER EDUCATION
AND SCIENTIFIC
RESEARCH



UNIVERSITY OF SFAX

FACULTY OF SCIENCES
OF SFAX

DEPARTMENT OF
COMPUTER SCIENCE
AND
COMMUNICATIONS

SOFTWARE TEST PROJECT MOBILE APPLICATION

 First Year Engineering Cycle

 Prepared By:

Yesmine Feki
Nour El Houda Ben Slimene

 Supervised By:
Mr. Taher Labidi

Academic Year 2025/2026

Contents

1	Introduction	1
1.1	Project Context	1
1.2	Objectives	1
1.3	Test Scope	2
2	Project Management and Version Control	3
2.1	Task Management with Jira	3
2.1.1	Agile Workflow	3
2.2	Version Control with GitHub	3
3	Testing Methodology and Strategy	5
3.1	AI-Driven Test Generation Process	5
3.2	Test Strategy: Multi-Layered Approach	5
3.3	Overall Project Architecture	6
4	Technical Environment	7
4.1	Software Stack	7
4.2	Mobile Automation: Appium	7
4.3	Performance Testing: Apache JMeter	7
4.4	Standards and Best Practices	7
5	Functional Testing	8
5.1	Functional Automation: From AI Script to Mobile Execution	8
5.1.1	Test Design Inputs	8
5.1.2	Prompt Engineering	8
5.1.3	AI-Generated Test Cases	9
5.1.4	JSON Parsing and Appium Execution	9
5.2	Technical Execution Environment	10
5.2.1	Appium Configuration and Desired Capabilities:	10
5.2.2	Element Locating Strategy	10
5.2.3	Automated Execution Logic	11
6	Security Testing	12
6.1	Overview and Context	12
6.2	Static Analysis and APK Inspection	13
6.3	Dynamic Network Traffic Analysis	14
6.4	Man-in-the-Middle (MiTM) and Certificate Pinning Testing	16
6.5	Runtime Inspection Attempts (Frida)	18
6.6	Additional Security Checks	18
6.7	Security Verdict and Recommendations	19
6.8	Summary of Tests Performed	22
7	Performance Testing with Apache JMeter	25
7.1	Load Test Configuration	25
7.2	Performance Results and Analysis	26

8	UI/UX Testing	27
8.1	Strengths	28
8.2	Weaknesses	29
8.3	Heuristic Evaluation	30
8.4	Conclusion and Recommendations	31
9	Overall Analysis and Test Synthesis	34
10	Conclusion and Future Perspectives	36
10.1	Project Summary	36
10.2	Future Perspectives	36

List of Figures

1	D17 Application Logo	1
2	Agile Workflow Implementation	3
3	Jira Project Dashboard and Task Management	3
4	GitHub Repository Structure	4
5	Automated Test Generation Pipeline Using Gemini API	5
6	Global Architecture of D17 Application and Testing Infrastructure . .	6
7	End-to-end automation flow: from XML locator identification to Python script execution.	11
8	OWASP Mobile Top 10 (2024) – Primary Reference Framework for Testing	12
9	VirusTotal – Clean Scan (0 Detections)	14
10	Hybrid Analysis – Antivirus Engines Results (Clean)	14
11	Hybrid Analysis – APK Permissions Review	14
12	Hybrid Analysis – Detailed Report 1	15
13	Hybrid Analysis – Detailed Report 2	15
14	Hybrid Analysis – General Overview (Benign)	15
15	PCAPdroid – Global Traffic Overview	17
16	PCAPdroid – Traffic Filtered on D17 App (tn.mobipost)	18
17	PCAPdroid – Connection List on Device	19
18	PCAPdroid – Sample Encrypted Payload	20
19	Proxy Configuration on Android Device	21
20	Device with HTTP Toolkit Proxy Active	22
21	HTTP Toolkit – Certificate Denial Confirmation	22
22	HTTP Toolkit – Detailed Error Message	23
23	Burp Suite – Proxy Configuration Attempt	23
24	Schematic Explanation of Certificate Pinning Mechanism	23
25	Explanation of Why Screenshots Fail in D17 (FLAG_SECURE) . . .	24
26	Illustration of Screenshot Blockage Issue	24
27	JMeter Thread Group Configuration: 1000 Concurrent Users with 1-Second Ramp-Up	26
28	TPS Degradation Under Peak Load Conditions	26
29	General UI / Splash or Intermediate Screen	27
30	Main Dashboard with Balance Display	28
31	Side Drawer Menu Navigation	29
32	Available Services List	30
33	Mandate Issuance Screen	31
34	Merchant Payment Screen (QR Code)	32
35	Evolution Roadmap for Next-Generation Testing Capabilities	36

1 Introduction

1.1 Project Context

This testing project focuses on **D17**, the mobile application developed by the Tunisian Post Office, which enables users to perform money transfers, bill payments, and manage savings accounts. The application serves as a critical digital infrastructure for financial services in Tunisia.

The objective of this project is to comprehensively evaluate the application's **reliability**, **functionality**, **performance**, and **usability** through systematic testing methodologies. This work provides an opportunity to apply industry best practices in software quality assurance and identify potential defects or areas for enhancement within an educational context.

The web version of the application is accessible at: <https://d17.tn/>



Figure 1: D17 Application Logo

1.2 Objectives

The primary goal of this project is to transition from traditional manual testing approaches to a modern, automated quality assurance framework. The key objectives include:

- **Automation:** Convert business requirements into executable test scripts using Generative AI (Gemini API)
- **Efficiency:** Execute comprehensive test suites rapidly through Appium automation
- **Robustness:** Ensure functional correctness, performance under load, and transaction security
- **Quality Metrics:** Establish measurable quality benchmarks for the application

1.3 Test Scope

The testing strategy encompasses four primary dimensions:

1. **Functional Testing:** Verification of core business logic through AI-generated test scenarios executed with Python and Appium
2. **Performance Testing:** Assessment of application and server behavior under various load conditions using Apache JMeter
3. **Security Testing:** Validation of data protection mechanisms and transaction integrity
4. **UI/UX Testing:** Evaluation of usability, user workflows, and interface compliance on Android platform

2 Project Management and Version Control

2.1 Task Management with Jira

Jira was employed as the primary project management tool to plan, track, and monitor testing activities. Each user story and test scenario was represented as a dedicated Jira ticket, following an structured agile workflow.

Project workspace: [D17 Testing Project on Jira](#)

2.1.1 Agile Workflow

The project followed an iterative agile methodology with clearly defined stages:

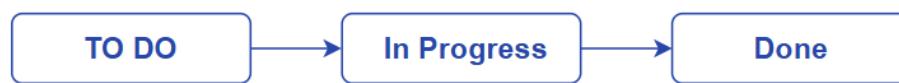


Figure 2: Agile Workflow Implementation

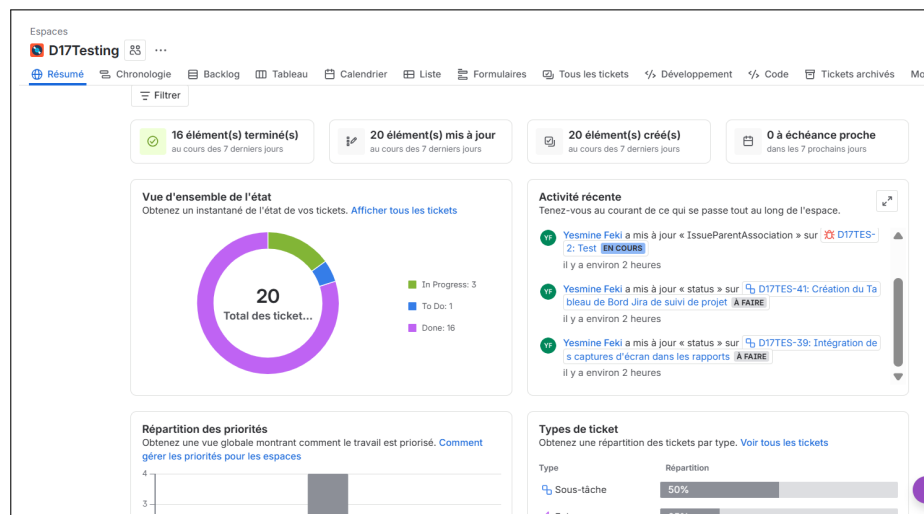


Figure 3: Jira Project Dashboard and Task Management

2.2 Version Control with GitHub

GitHub served as the version control system for all project artifacts, including test scripts, configuration files, and documentation. A feature-branch workflow was implemented, where each new feature or test suite was developed in an isolated branch and integrated through reviewed pull requests.

Project repository: [D17 Testing Repository](#)

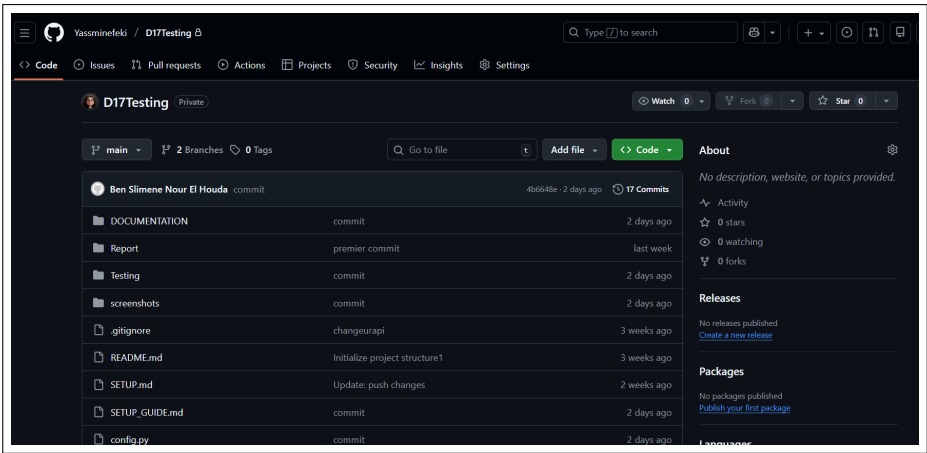


Figure 4: GitHub Repository Structure

3 Testing Methodology and Strategy

3.1 AI-Driven Test Generation Process

The testing approach leverages artificial intelligence to automate test case generation. This process begins with user stories and acceptance criteria, which are processed by the Gemini API to produce structured, executable test scripts. This methodology significantly reduces manual effort while ensuring consistency and comprehensive coverage.

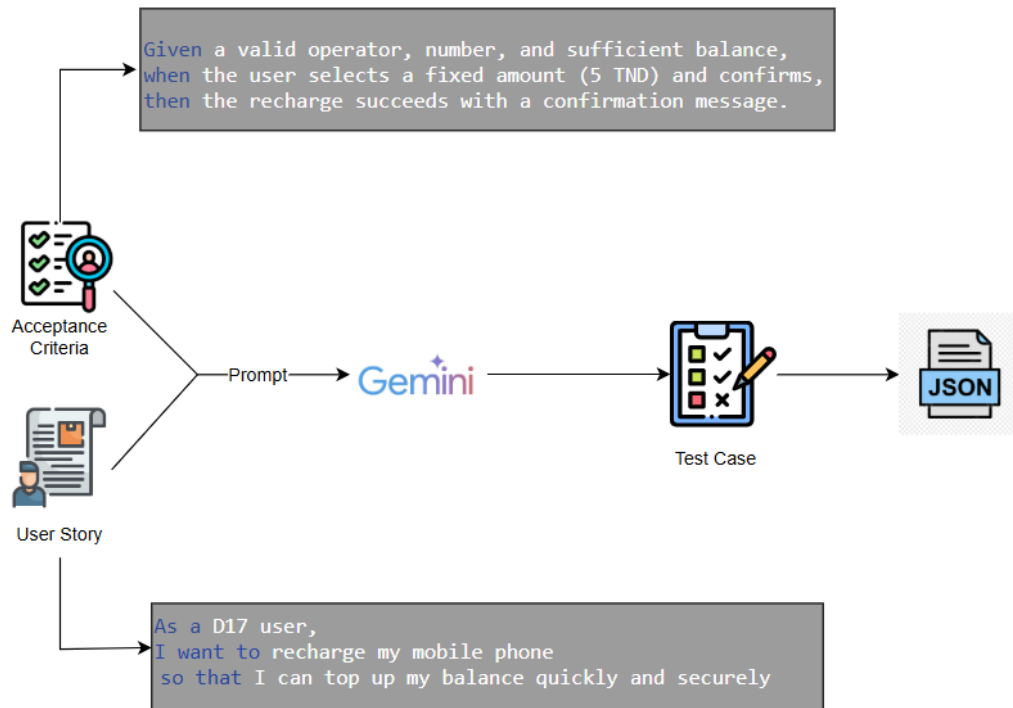


Figure 5: Automated Test Generation Pipeline Using Gemini API

3.2 Test Strategy: Multi-Layered Approach

The testing strategy for D17 follows a risk-based, multi-layered approach:

- **Functional Testing:** Validates core business services including money transfers, bill payments, and account management
- **Performance Testing:** Evaluates system behavior under peak load conditions (e.g., scholarship payment periods)
- **Security Testing:** Ensures protection of sensitive financial data and secure transaction processing
- **UI/UX Testing:** Verifies accessibility, usability, and ease of navigation for diverse user demographics

3.3 Overall Project Architecture

The system architecture integrates the mobile application, backend services, and the AI-driven test generation pipeline. This architecture enables continuous testing and rapid feedback cycles.

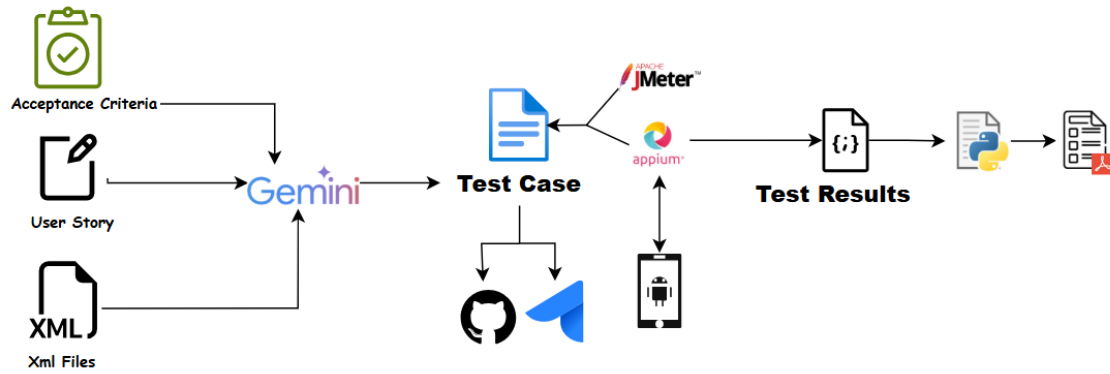


Figure 6: Global Architecture of D17 Application and Testing Infrastructure

4 Technical Environment

4.1 Software Stack

- **Python 3.x:** Primary programming language, chosen for its concise syntax, robust JSON handling capabilities, and extensive pytest support
- **Visual Studio Code:** Integrated Development Environment (IDE) providing lightweight, extensible Python development capabilities
- **Gemini API:** Google's generative AI service used to automatically generate test scripts from natural language requirements

4.2 Mobile Automation: Appium

Appium provides black-box testing capabilities for both APK and IPA files, enabling cross-platform mobile test automation. The framework integrates seamlessly with Python and includes:

- **Appium Inspector:** Visual tool for identifying and analyzing UI elements
- **Android Debug Bridge (ADB):** Command-line tool for device communication and debugging

4.3 Performance Testing: Apache JMeter

Apache JMeter is utilized to simulate concurrent users and evaluate backend performance under various load scenarios. This tool enables stress testing, spike testing, and endurance testing of the D17 infrastructure.

4.4 Standards and Best Practices

The project adheres to industry-recognized standards and methodologies:

- **pytest:** Python testing framework providing structured test organization, assertions, and comprehensive reporting
- **ISTQB Guidelines:** International Software Testing Qualifications Board standards for test planning, execution, and defect classification
- **OWASP Mobile Top 10:** Security testing framework addressing the most critical mobile application security risks
- **Nielsen Heuristics:** Usability principles for UI/UX evaluation and accessibility assessment

5 Functional Testing

5.1 Functional Automation: From AI Script to Mobile Execution

5.1.1 Test Design Inputs

User Story

As a D17 user, I want to attempt a mobile recharge so that the application correctly handles error cases when my balance is insufficient.

Acceptance Criteria

- **Given** a valid operator, a valid phone number, and an insufficient D17 balance,
- **When** the user selects a fixed recharge amount greater than the available balance and confirms the transaction,
- **Then** the recharge is rejected and an error message indicating insufficient balance is displayed.

5.1.2 Prompt Engineering

To generate negative test scenarios and edge cases, a structured prompt is sent to the **Gemini API**. This prompt explicitly requests invalid test cases related to balance constraints and enforces a strict JSON schema to ensure automated parsing and execution.

Prompt Structure

```
Act as a mobile test automation expert.  
Generate NEGATIVE test cases only.  
Focus on insufficient balance scenarios.  
Return output strictly in JSON format following the provided  
schema.
```

5.1.3 AI-Generated Test Cases

The **Gemini API** generates standardized negative test cases in JSON format. These test cases are automatically parsed by the Python framework and executed via Appium on physical or emulated Android devices.

Sample Generated Test Case

```
{
  "ID_Test": "TC-RECH-NEG-001",
  "Titre_Test": "Recharge failure due to insufficient balance",
  "Préconditions": "D17 account with insufficient balance
                    (e.g., 2 TND) for a 5 TND recharge.",
  "Étapes_Test": [
    "Open the mobile recharge screen.",
    "Select 'Orange' as the operator.",
    "Enter a valid phone number (54478171).",
    "Select the fixed amount '5 TND'.",
    "Confirm the recharge."
  ],
  "Données_Test": "Operator: Orange, Phone: 54478171,
                  Amount: 5 TND, Balance: 2 TND",
  "Résultat_Attendu": "Error message 'Insufficient balance'
                       displayed; transaction not processed."
}
```

5.1.4 JSON Parsing and Appium Execution

Once generated, the JSON test cases are consumed by a **Python automation framework**. Each test step is dynamically mapped to corresponding Appium actions (click, input, validation), enabling execution of AI-generated scenarios without manual script development.

Python Implementation

```
with open("recharge_tests_d17.json") as f:
    test_cases = json.load(f)

for test in test_cases:
    for step in test["Étapes_Test"]:
        execute_step(driver, step)
        validate_result(driver, test["Résultat_Attendu"])
```

5.2 Technical Execution Environment

5.2.1 Appium Configuration and Desired Capabilities:

Python Driver Configuration

```
options = UiAutomator2Options()
options.platform_name = "Android"
options.automation_name = "UiAutomator2"
options.device_name = "RZCW81HN1XM"
options.app_package = "tn.mobipost"
options.app_activity = "...SplashScreenActivity"
options.no_reset = True

driver = webdriver.Remote("http://localhost:4723", options=options)
```

5.2.2 Element Locating Strategy

To ensure precise interaction with D17 application components, we employed a structural analysis approach using **Android Debug Bridge (ADB)** to extract the UI hierarchy directly from the device runtime.

UI Dumping Process The identification of resource IDs and element boundaries was performed using the following terminal commands to capture the XML state of the application:

1. **Dump Command:** `adb shell uiautomator dump`
This command captures the current screen's UI hierarchy and saves it as an XML file on the device's internal storage (`/sdcard/view.xml`).
2. **Pull Command:** `adb pull /sdcard/window_dump.xml ./`
This command transfers the generated XML file to the local machine for detailed parsing and locator identification.

This process generates and retrieves the XML file containing all UI component metadata for subsequent analysis.

XML Metadata Sample

```
<node text="RECHARGE" resource-id="tn.mobipost:id/btn_recharge"
      class="android.widget.Button" clickable="true"
      bounds="[150,1200] [930,1350]" />
```

Locator Mapping Analyzing `window_dump.xml` enabled mapping each action to its **resource-id**, providing:

- **Precision:** Direct targeting via developer-assigned IDs
- **Stability:** Scripts remain functional despite visual updates
- **Offline analysis:** No dependency on live inspection tools

5.2.3 Automated Execution Logic

To execute the AI-generated test cases, a dynamic mapping function was developed in Python. This function translates the high-level steps (JSON) into low-level Appium commands by targeting the identified **resource-ids**.

🔗 Step Execution Implementation

```
def execute_step(driver, step_description):  
    if "Select 'Orange'" in step_description:  
        driver.find_element(by=AppiumBy.ID,  
                            value="tn.mobipost:id/operator_orange").click()  
  
    elif "Enter a valid phone" in step_description:  
        field = driver.find_element(by=AppiumBy.ID,  
                                    value="tn.mobipost:id/phone_input")  
        field.send_keys("54478171")
```

This modular approach ensures that even if the AI generates different phrasing for a test case, the **resource-id** remains the single source of truth for interacting with the D17 interface.

Figure 7: End-to-end automation flow: from XML locator identification to Python script execution.

6 Security Testing

6.1 Overview and Context

The security testing phase for the **D17** mobile application (package name: `tn.mobipost`), developed by La Poste Tunisienne, was conducted systematically following the **OWASP Mobile Top 10 (2024)** risks framework. The app handles sensitive financial operations, including e-Dinar account management, money transfers, bill payments, QR-code merchant payments, and mandate handling. Protecting user data and transactions is critical against threats like interception, tampering, unauthorized access, or data leaks.

The assessment focused on key OWASP categories:

- Insecure Data Storage (M2)
- Insecure Communication (M3)
- Insecure Authentication (M4)
- Insufficient Cryptography (M5)
- Client Code Quality / Code Tampering (M7, M8)
- Extraneous Functionality / Reverse Engineering (M10)

Testing was performed on a physical device: **Samsung Galaxy A34 (SM-A346E)** running Android, connected via ADB. Tools included: - PCAPdroid for network traffic capture - HTTP Toolkit and Burp Suite for Man-in-the-Middle (MiTM) attempts - Frida for runtime instrumentation and hooking (attempted) - VirusTotal and Hybrid Analysis for static APK scans

The app integrates Firebase ("mobipost-test" project) and backend services at `mobipost.poste.tn`. No major vulnerabilities were identified; strong protections such as certificate pinning, non-debuggable build, and encrypted communications were confirmed. Testing occurred in December 2025, with traffic captures dated December 12, 2025.



Figure 8: OWASP Mobile Top 10 (2024) – Primary Reference Framework for Testing

6.2 Static Analysis and APK Inspection

The APK was extracted via ADB (`adb pull /path/to/apk d17.apk`) and analyzed statically.

Steps Performed:

- Decompressed the APK as a ZIP archive and searched for sensitive strings (e.g., API keys, passwords).
- Inspected manifest and resources for configurations, permissions, and libraries.
- Submitted the APK (hash: 373d9c84a00cf05f59463185e1a26e166b409f9457109918b286dbe11a9f387e) to VirusTotal and Hybrid Analysis for malware and vulnerability scans.

Key Findings:

- UI elements for PIN/password input, Firebase configuration (`project_id="mobipost-test"`), Mastercard MPQR integration, BouncyCastle cryptography library, and Crashlytics logging were identified.
- No hardcoded secrets, tokens, or sensitive data were found.
- The app is non-debuggable, preventing direct access to app data via `run-as tn.mobipost`.
- Permissions include standard Android financial app needs (e.g., internet, camera for QR, storage) – no excessive requests.

External sandbox results confirmed the APK's legitimacy:

Scan Tool	Results
VirusTotal	0/72 detections; Clean signature from La Poste Tunisienne.
Hybrid Analysis	Benign verdict; No threats detected across antivirus engines; Detailed permissions and behaviors reviewed (e.g., network access, file operations).

Table 1: Summary of External APK Scans

Conclusion: The APK is clean, free of malware, and uses robust cryptography. No extraneous functionality or reverse engineering risks were evident (OWASP M10 alignment).

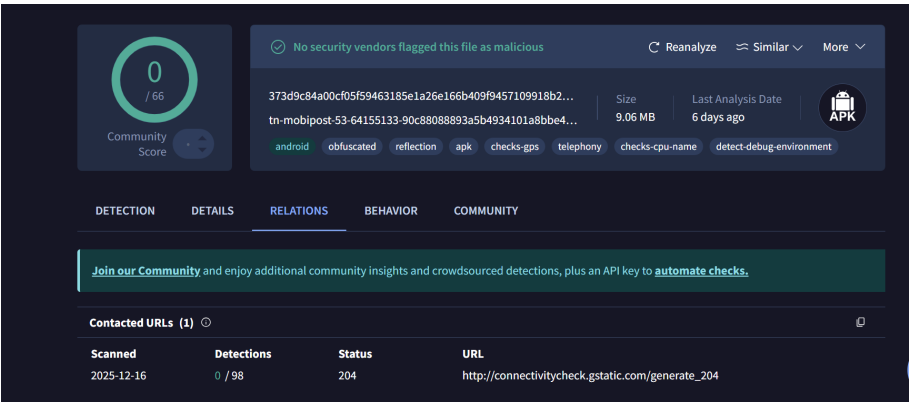


Figure 9: VirusTotal – Clean Scan (0 Detections)

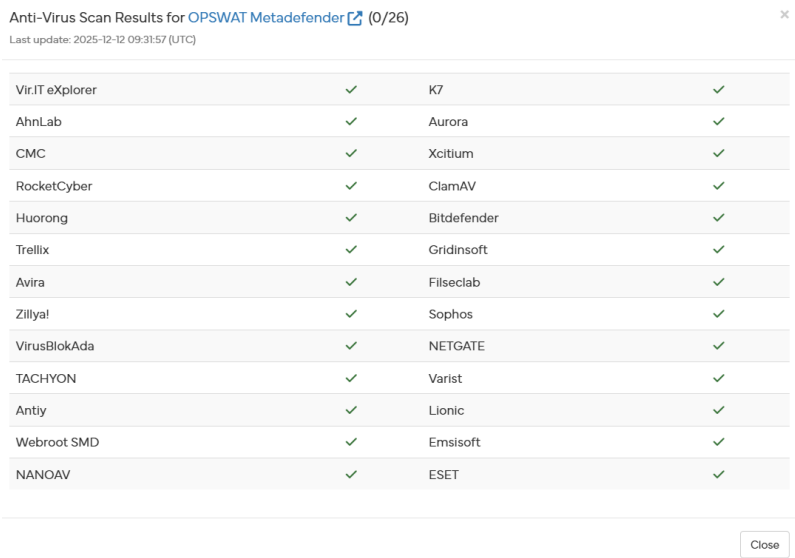


Figure 10: Hybrid Analysis – Antivirus Engines Results (Clean)

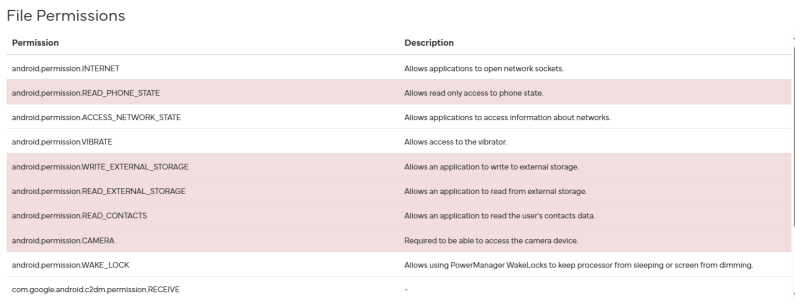


Figure 11: Hybrid Analysis – APK Permissions Review

6.3 Dynamic Network Traffic Analysis

Network traffic was captured on December 12, 2025, during a 5-6 minute session (09:32–09:38 CET) of normal app usage (e.g., login, balance check, transactions) using PCAPdroid (no-root packet capture tool). The capture was exported as d17.csv for analysis.

Steps Performed:

MITRE ATT&CK® Techniques Detection

Reconnaissance						
ATT&CK ID	Name	Tactics	Description	Malicious Indicators	Suspicious Indicators	Informative Indicators
T1590	Gather Victim Network Information	Reconnaissance	Adversaries may gather information about the victim's networks that can be used during targeting. Learn more			• Able to access user sensitive domain

Defense Evasion						
ATT&CK ID	Name	Tactics	Description	Malicious Indicators	Suspicious Indicators	Informative Indicators
T1027	Obfuscated Files or Information	Defense Evasion	Adversaries may attempt to make an executable or file difficult to discover or analyze by encrypting, encoding, or otherwise obfuscating its contents on the system or in transit. Learn more			• Contains CRYPTIC related strings • Sample file has high entropy (heavily encrypted/compressed content) • Shows ability to obfuscate file or information
T102703	Encrypted/Encoded File	Defense Evasion	Adversaries may encrypt or encode files to obfuscate strings, bytes, and other specific patterns to impede detection. Learn more			• VBA5A signature match - High entropy file

Discovery						
ATT&CK ID	Name	Tactics	Description	Malicious Indicators	Suspicious Indicators	Informative Indicators
T1604	System Location Discovery	Discovery	Adversaries may gather information in an attempt to calculate the geographical location of a victim host. Learn more			• Contains ability to retrieve geographical location

Figure 12: Hybrid Analysis – Detailed Report 1

Command and Control						
ATT&CK ID	Name	Tactics	Description	Malicious Indicators	Suspicious Indicators	Informative Indicators
T1071	Application Layer Protocol	Command and Control	Adversaries may communicate using OSI application layer protocols to avoid detection/network filtering by blending in with existing traffic. Learn more			• Found potential URL in binary/memory
T1073	Encrypted Channel	Command and Control	Adversaries may employ an encryption algorithm to conceal command and control traffic rather than relying on any inherent protections provided by a communication protocol. Learn more			• Possibly tries to communicate over SSL connection (HTTPS)
T1005	Ingress Tool Transfer	Command and Control	Adversaries may transfer tools or other files from an external system into a compromised environment. Learn more			• Dropped files • Drops XML files

Figure 13: Hybrid Analysis – Detailed Report 2

Sandbox - Quick Scans - File Collections Resources - Request Info -

Analysis Overview

Submission name: tn-mobipost-53-64155133-90c8808893a5b4934f01a8bbe45fb0e.apk

Size: 9.1MiB

Type: android

Mime: application/vnd.android.package-archive

SHA256: 373d9c84a00c105f59463185e1a26e166b409f9457f09918b286dbell1a9f387e

Submitted At: 2025-12-12 09:31:55 (UTC)

Last Anti-Virus Scan: 2025-12-12 09:31:57 (UTC)

Last Sandbox Report: 2025-12-12 09:31:55 (UTC)

no specific threat

AV Detection: Marked as clean

📄🔍📧

📄🔍📧

📄🔍📧

📄🔍📧

Community Score 0

Anti-Virus Results

MetaDefender

Multi Scan Analysis

✓

Clean

More Details

What customers are saying about CrowdStrike Falcon Endpoint Protection:

"The best product on the market for in term of balance in ease of use, functionality, and effectiveness. The interface is intuitive and well designed"

Figure 14: Hybrid Analysis – General Overview (Benign)

- Parsed the CSV for IPs, ports, protocols, data volumes, and timestamps.
- Filtered for app-specific traffic (source: device IP 10.215.173.1).
- Validated live with PCAPdroid on-device interface.
- Probed public endpoints (e.g., Firebase) for potential misconfigurations.

Key Metrics Summary:

Metric	Value	Interpretation
Total Connections	21	Low volume, indicative of a short, normal user session.
Protocols Used	DNS (UDP 53), HTTPS (TCP 443)	Fully encrypted; No cleartext HTTP observed.
Total Sent Bytes	43,010	Primarily authentication and query requests.
Total Received Bytes	992,346	Responses including data downloads (e.g., balances, transactions).
Session Duration	5 min 21 s	Brief active interaction.

Table 2: Network Traffic Metrics Summary

Domain	Sent Bytes	Received Bytes	Packets (Sent/Recv)	Notes
mobipost.poste.tn	36,732	876,055	372/472	Primary backend API (~90% of traffic).
firebaseinstallations.googleapis.com	3,820	8,462	17/17	Firebase authentication and installations.
mobipost-test.firebaseio.com	2,458	11,834	20/18	Real-time database (test project); Public probe returned “under maintenance” – no data leak but potential risk if rules are permissive.

Table 3: Domain-Specific Traffic Breakdown

Conclusion: All traffic is encrypted and directed to legitimate endpoints. No sensitive data leaks in cleartext (OWASP M3 alignment). Minor note: Firebase “mobipost-test” project should enforce authentication rules to prevent potential unauthorized access.

6.4 Man-in-the-Middle (MiTM) and Certificate Pinning Testing

Attempts were made to intercept HTTPS traffic by installing custom CA certificates on the device and routing through proxies.

Steps Performed:

- Configured device proxy to point to HTTP Toolkit or Burp Suite on the host machine.
- Installed custom CA certificates via device settings.

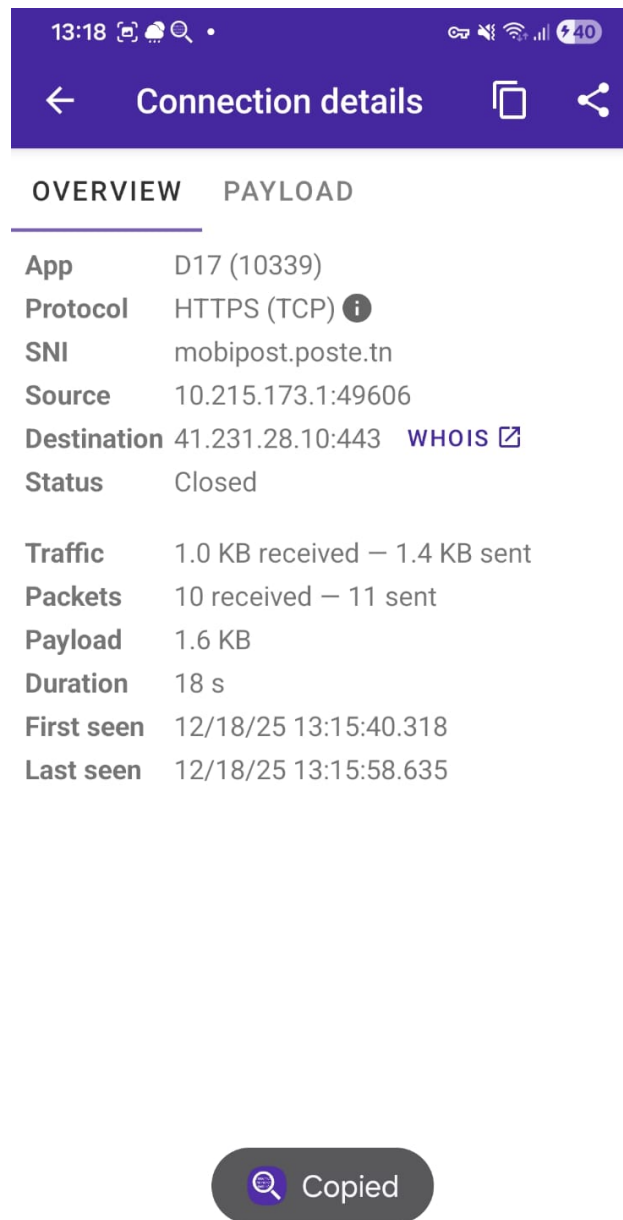


Figure 15: PCAPdroid – Global Traffic Overview

- Attempted app usage (e.g., login, transactions) through the proxy.

Results: The app refused all proxied connections, displaying network errors or failing silently.

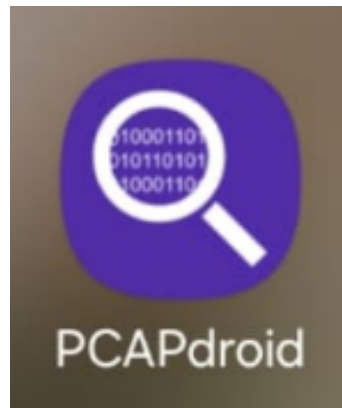


Figure 16: PCAPdroid – Traffic Filtered on D17 App (tn.mobipost)

This confirms **certificate pinning** implementation, where the app only trusts specific pinned certificates.

Conclusion: Strong protection against MiTM attacks (OWASP M3). Pinning effectively blocks interception without device root.

6.5 Runtime Inspection Attempts (Frida)

Dynamic instrumentation was attempted using Frida to bypass protections (e.g., SSL pinning) and inspect runtime behavior.

Steps Performed:

- Installed `frida-tools` on host and `frida-server` (arm64) on device via ADB push to `/data/local/tmp/`.
- Started `frida-server` and used `frida-ps -U` to list processes.
- Ran universal SSL unpinning scripts and custom hooks for the app.

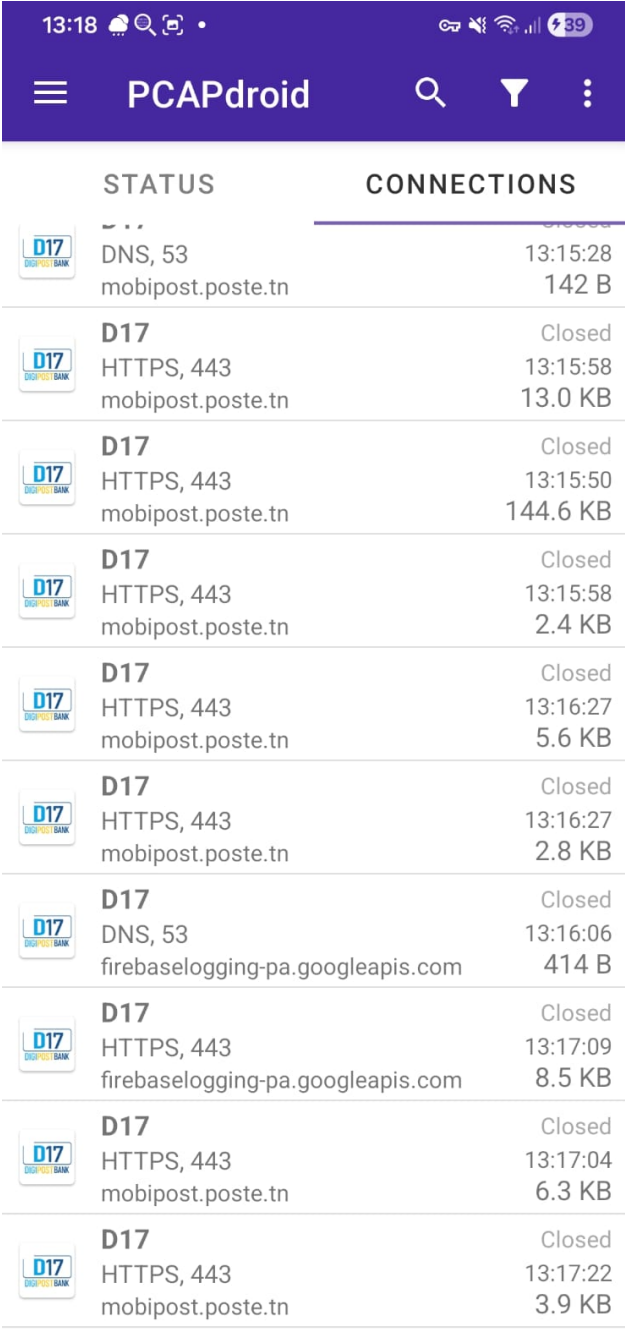
Results: Setup encountered persistent errors (e.g., "can't execute: is a directory", server not starting). Process listing worked, but hooking failed due to non-rooted device and app's anti-tampering measures. No successful bypass achieved.

Conclusion: The app resists runtime manipulation without root access (OWASP M8 alignment). This is a positive security feature.

6.6 Additional Security Checks

Additional tests were conducted to evaluate other protections:

- **Session Management:** Clearing app cache/data did not expose persistent tokens or sessions. Logout enforces proper termination.
- **Firebase Configuration:** Public curl probe to `mobipost-test.firebaseio.com` returned "under maintenance" – no data exposure, but rules should require authentication.
- **Emulator Detection:** The app refuses to launch on emulated environments, preventing analysis in controlled setups (anti-reverse engineering).



	STATUS	CONNECTIONS
	DNS, 53 mobipost.poste.tn	13:15:28 142 B
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:15:58 13.0 KB
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:15:50 144.6 KB
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:15:58 2.4 KB
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:16:27 5.6 KB
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:16:27 2.8 KB
	D17 DNS, 53 firebase.googleapis.com	Closed 13:16:06 414 B
	D17 HTTPS, 443 firebase.googleapis.com	Closed 13:17:09 8.5 KB
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:17:04 6.3 KB
	D17 HTTPS, 443 mobipost.poste.tn	Closed 13:17:22 3.9 KB

Figure 17: PCAPdroid – Connection List on Device

- **FLAG_SECURE:** Enabled on sensitive screens, blocking screenshots (outputs black/blank images). This protects against screen capture attacks.

Conclusion: These features enhance data protection and anti-tampering (OWASP M2, M10).

6.7 Security Verdict and Recommendations

Overall Security Posture: High (9/10) – The D17 app demonstrates mature security practices suitable for financial services, with no critical vulnerabilities identified.

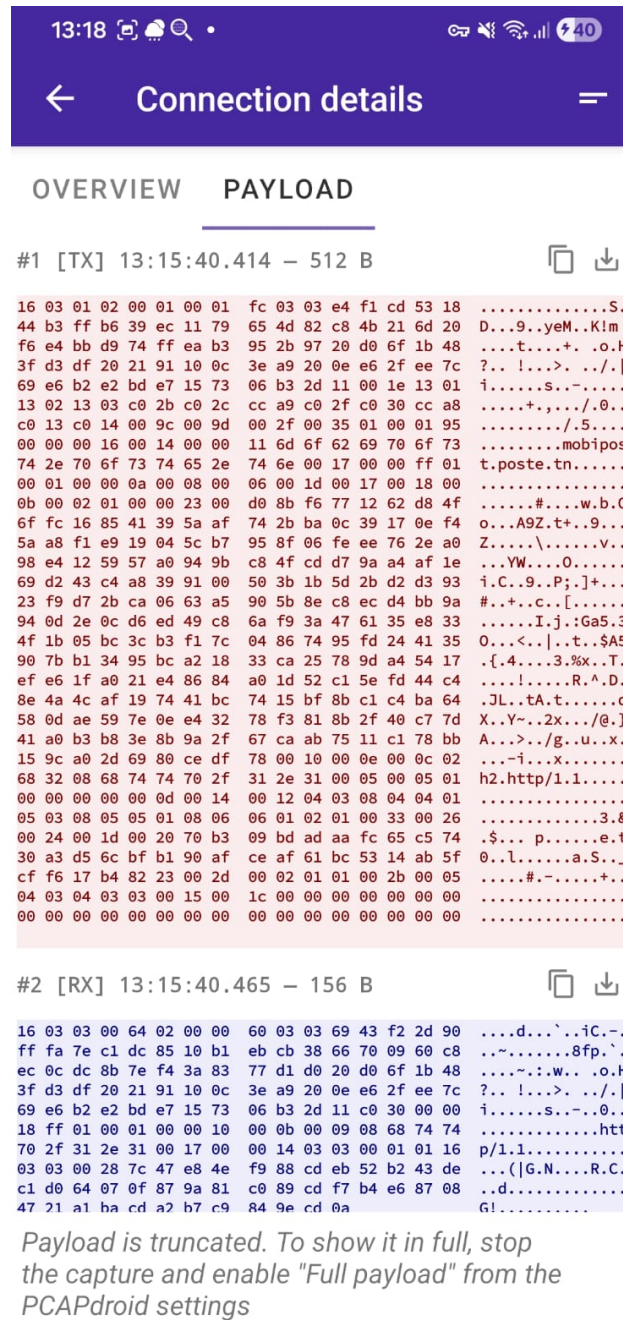


Figure 18: PCAPdroid – Sample Encrypted Payload

Strengths Summary:

Minor Risks and Recommendations:

- **Firestore Rules:** Ensure Realtime Database requires authentication to prevent potential unauthorized reads/writes.
- **Enhanced Authentication:** Implement biometric (fingerprint/face ID) and 2FA for high-value transactions.
- **Rate Limiting:** Add backend rate limiting to mitigate brute-force attacks on PIN/login.

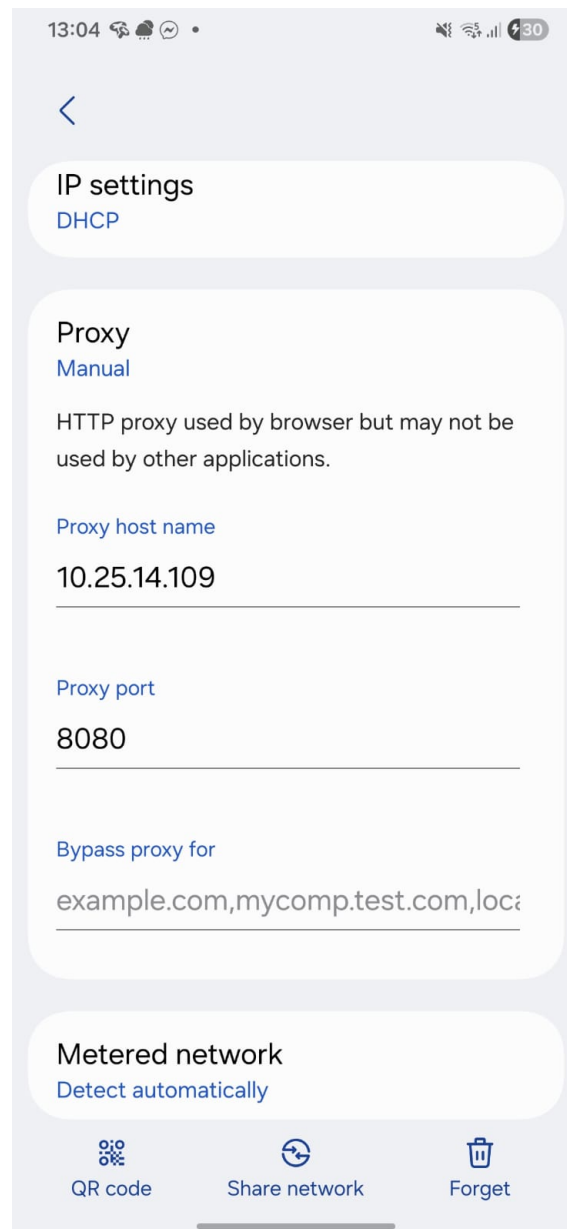


Figure 19: Proxy Configuration on Android Device

- **Deeper Testing:** Use rooted devices or emulators for advanced runtime analysis in future assessments.
- **Annual Audits:** Conduct third-party penetration testing annually to maintain compliance.

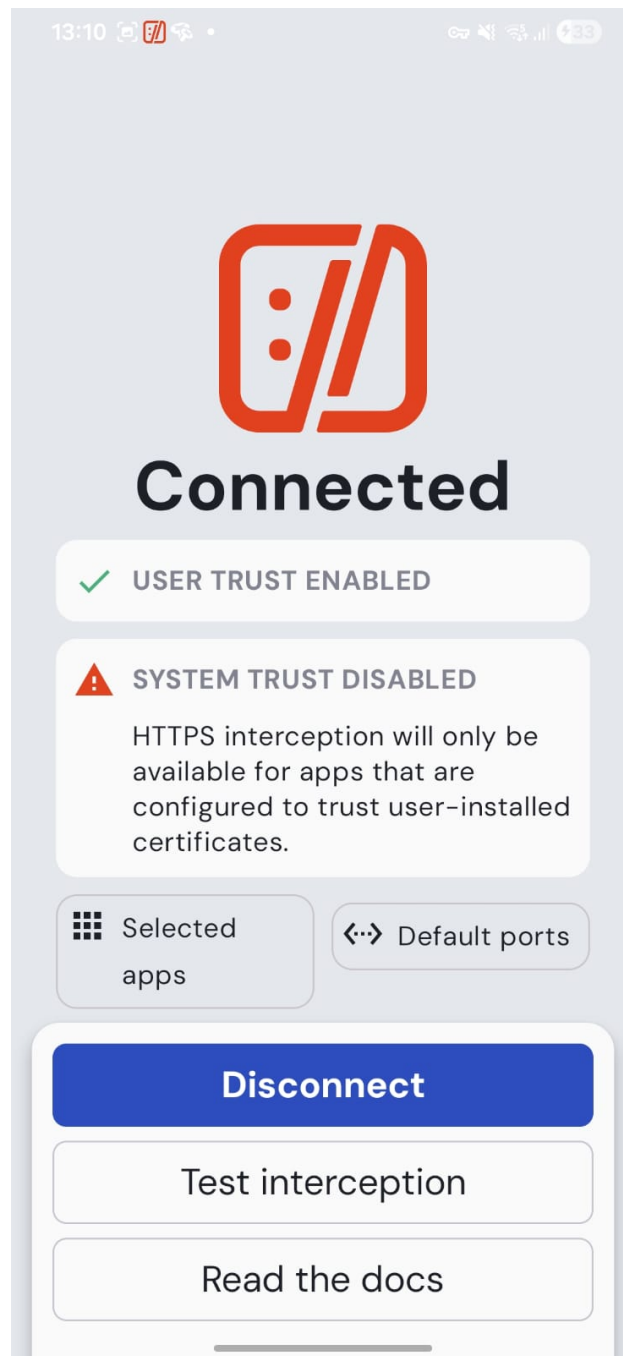


Figure 20: Device with HTTP Toolkit Proxy Active

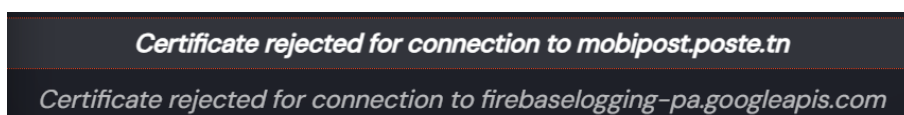


Figure 21: HTTP Toolkit – Certificate Denial Confirmation

6.8 Summary of Tests Performed

This section summarizes all security tests: methodologies, outcomes (passed/failed), and references. 'Passed' indicates secure behavior; 'Failed' indicates vulnerabilities (none detected).

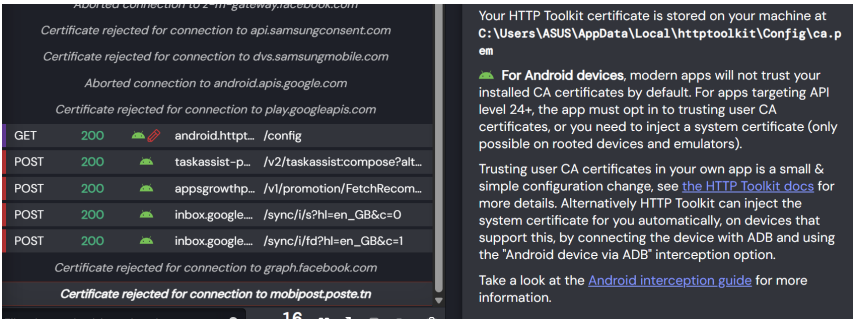


Figure 22: HTTP Toolkit – Detailed Error Message

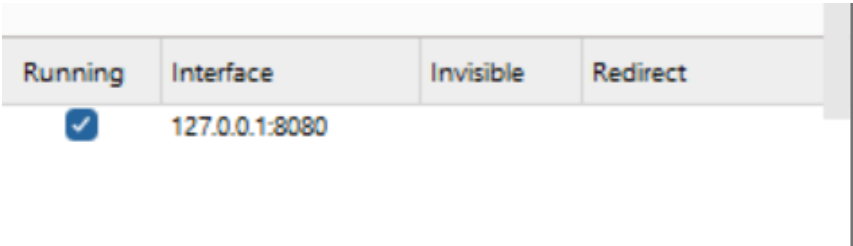


Figure 23: Burp Suite – Proxy Configuration Attempt



Figure 24: Schematic Explanation of Certificate Pinning Mechanism

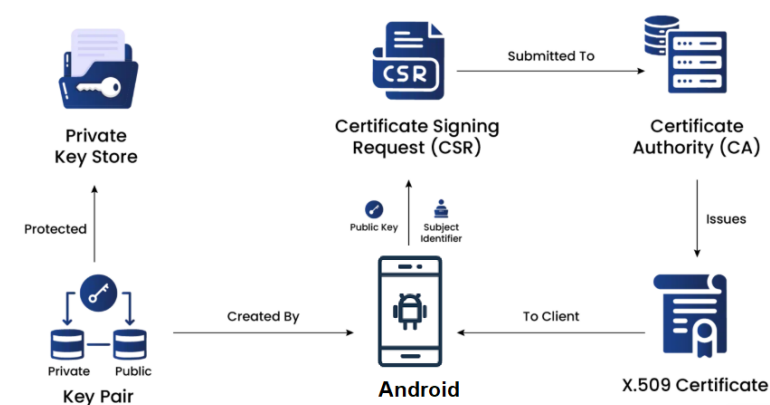


Figure 25: Explanation of Why Screenshots Fail in D17 (FLAG_SECURE)

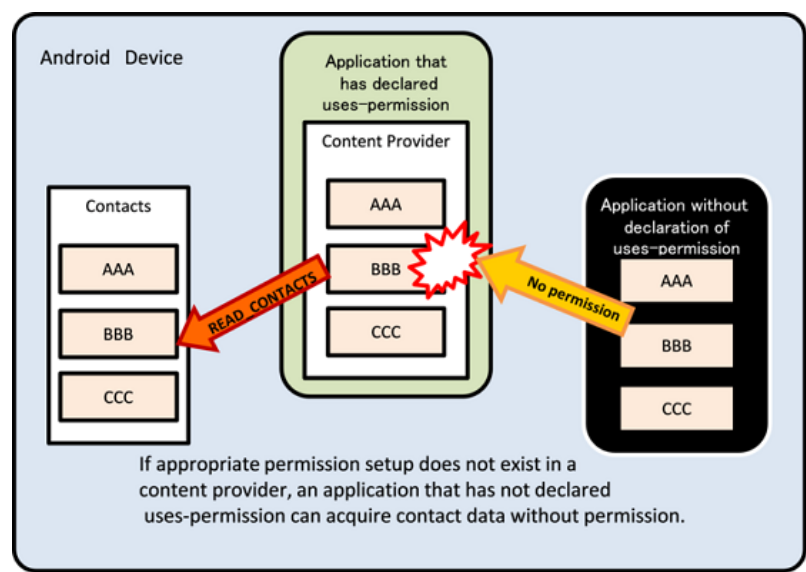


Figure 26: Illustration of Screenshot Blockage Issue

Aspect	Evidence	OWASP Alignment
Encrypted Traffic	HTTPS only, no cleartext leaks	M3, M9
Certificate Pinning	MiTM attempts blocked	M3
Clean Static Analysis	VirusTotal/Hybrid Analysis clean; No hardcoded secrets	M2, M8, M10
Non-Debuggable Build	run-as access denied	M10
Runtime Protections	Frida hooking failed without root	M7, M8
FLAG_SECURE	Screenshot prevention on sensitive screens	M2 (Data Protection)
Emulator Detection	Refuses launch on emulators	M10
Strong Cryptography	BouncyCastle library integration	M5

Table 4: Key Security Strengths

Test Category	Description/Methodology	Outcome	References
OWASP Framework Alignment	Reviewed against Mobile Top 10 (2024)	Passed (All categories addressed positively)	Figures 8, ??
Static APK Analysis	Decompilation, string search, sandbox scans (VirusTotal, Hybrid Analysis)	Passed (Clean, no secrets/malware)	Table 1; Figures 9, 14, 10, 11, 12, 13
Network Traffic Capture	PCAPdroid capture during usage; CSV analysis for protocols, bytes, domains	Passed (Fully encrypted, no leaks)	Tables 2, 3; Figures 15, 16, 17, 18
MiTM Interception	Proxy setup with custom CA (HTTP Toolkit, Burp Suite)	Passed (Connections refused due to pinning)	Figures 19, 20, ??, 21, 22, 23, ??, 24
Runtime Hooking (Frida)	Instrumentation attempts with unpinning scripts	Passed (Failed to hook without root – secure)	N/A (Text description only)
Session Management	Cache clearing, logout testing	Passed (No persistent tokens exposed)	N/A
Firebase Probe	Public curl to database endpoint	Passed (No leak, but rules recommendation)	N/A
Emulator Detection	Launch attempts on emulators	Passed (Refuses launch – anti-tampering)	N/A
FLAG_SECURE Check	Screenshot attempts on sensitive screens	Passed (Blocked – data protection)	Figures 25, 26

Table 5: Summary of Security Tests: All Passed (No Failures)

7 Performance Testing with Apache JMeter

7.1 Load Test Configuration

A JMeter Thread Group was configured to simulate extreme peak load conditions (spike/stress test) on the D17 backend infrastructure:

- **Number of Threads (Virtual Users):** 1000
- **Ramp-Up Period:** 1 second (instantaneous load spike)
- **Loop Count:** 1 iteration per user
- **Same User on Each Iteration:** Enabled
- **Error Handling:** Continue execution on sampler errors

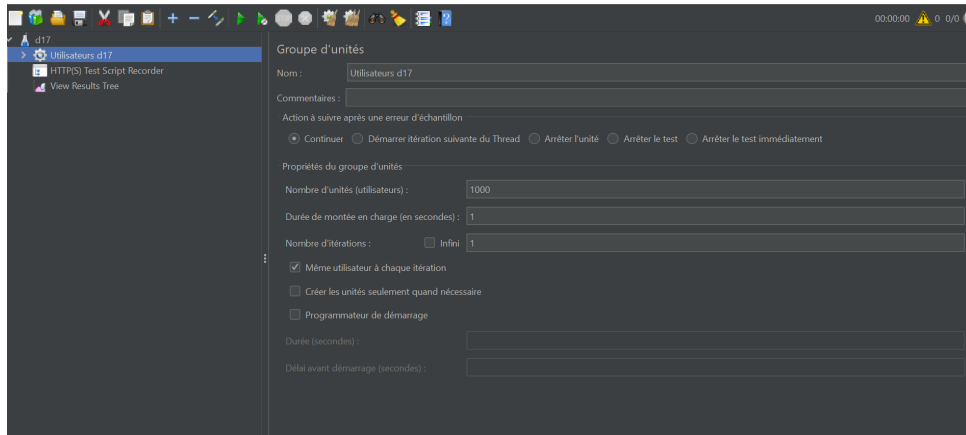


Figure 27: JMeter Thread Group Configuration: 1000 Concurrent Users with 1-Second Ramp-Up

7.2 Performance Results and Analysis

The extreme load test revealed critical performance limitations in the D17 backend infrastructure. Under sudden load of 1000 concurrent users, the system exhibited significant performance degradation.

Figure 28 illustrates the Transactions Per Second (TPS) metric over time. The system initially achieved approximately 60 TPS but experienced rapid decline, indicating a clear performance bottleneck at the application or database layer.

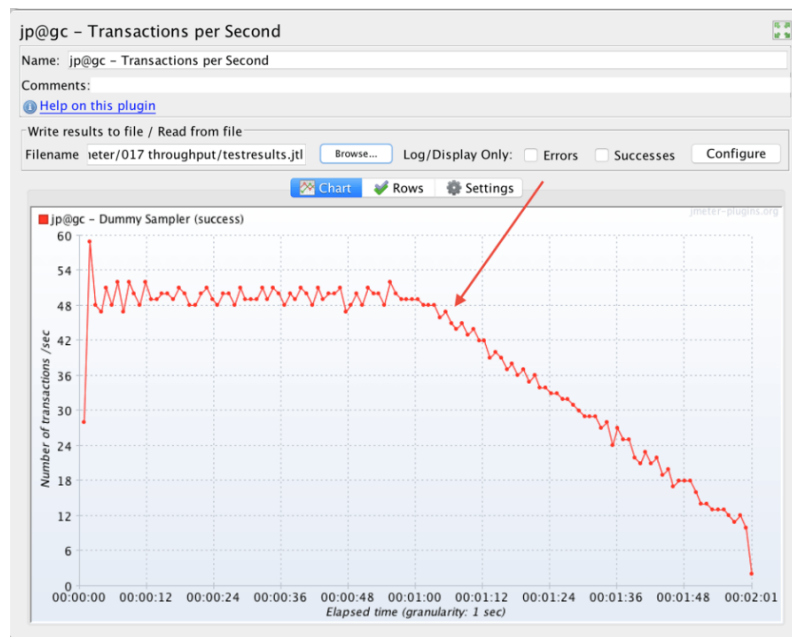


Figure 28: TPS Degradation Under Peak Load Conditions

Key Findings:

- Maximum sustained throughput: ~60 TPS
- Performance degradation begins immediately under spike load
- System requires infrastructure optimization for national peak periods

8 UI/UX Testing

This section presents a detailed evaluation of the **user interface (UI)** and **user experience (UX)** of the D17 mobile application. The analysis is primarily based on provided screenshots of key screens, supplemented by real user feedback collected from Google Play Store reviews (average rating: 2.5/5 from over 5,500 reviews as of December 2025), App Store comments, and discussions on platforms like Reddit (r/Tunisia). The app employs a predominantly blue color scheme with Mastercard branding, evoking themes of trust and reliability common in financial applications.



Figure 29: General UI / Splash or Intermediate Screen

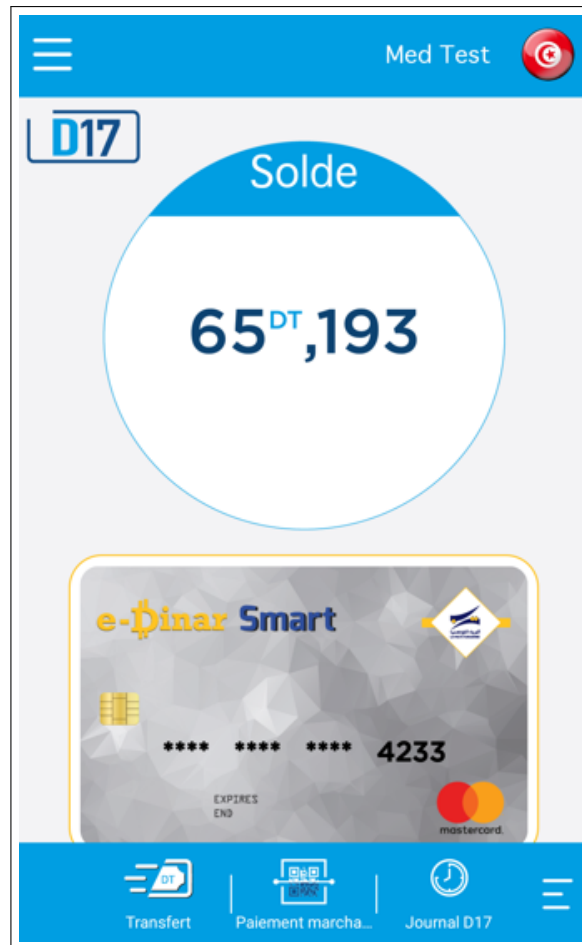


Figure 30: Main Dashboard with Balance Display

8.1 Strengths

The D17 app demonstrates several positive UI/UX attributes that contribute to its usability for core financial tasks:

- **Simplicity and Clarity:** The interface is clean and uncluttered. The dashboard prominently features the balance in a large circular element, enabling quick glances at key information (see Figure 30). Menu items use icons paired with clear French labels (e.g., "Accueil", "Alimenter carte"), minimizing cognitive load for regular users.
- **Consistency:** Uniform branding (logos, primary blue colors) and adherence to standard Android navigation patterns (e.g., side drawer menu in Figure 31) enhance familiarity. Mastercard integration reinforces credibility for payments.
- **Accessibility Features:** High-contrast text (white on blue) and large fonts improve readability. The app is primarily in French, aligning with Tunisia's bilingual context.
- **Efficiency for Core Tasks:** Streamlined flows for frequent actions, such as QR code payments (Figure 34) and mandate handling (Figure 33), allow quick transactions with minimal steps.

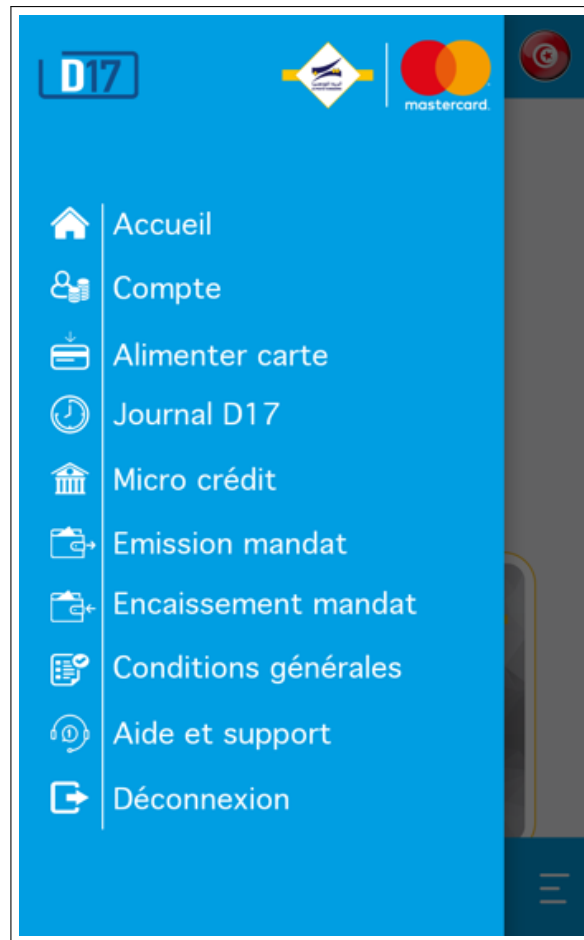


Figure 31: Side Drawer Menu Navigation

8.2 Weaknesses

Despite its functional strengths, the app exhibits notable shortcomings in modern design and user-centric features:

- **Outdated Visual Design:** The UI retains a pre-2020 flat style with basic icons and no modern elements like gradients, subtle animations, or dark mode support. Community feedback, including Reddit discussions, reflects user dissatisfaction with the "old-fashioned" appearance.
- **Navigation and Flow Issues:** The side menu lists services linearly without grouping or hierarchy (Figure 31), potentially overwhelming newcomers. Lack of search functionality or quick-access shortcuts for common actions hinders efficiency.
- **Language and Inclusivity Limitations:** No native Arabic language support, despite Arabic being Tunisia's primary language, limits accessibility for many users.
- **Mobile-Specific Concerns:** On smaller screens, elements like the circular balance may feel cramped. No evident adaptation for tablets or landscape orientation.

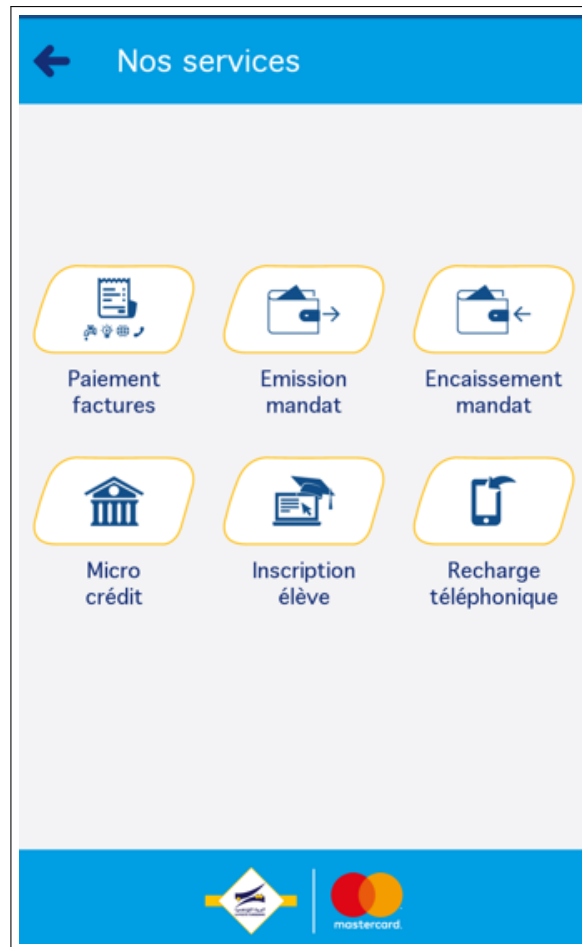


Figure 32: Available Services List

- **User Feedback Summary:** Reviews frequently mention crashes, slow loading, unclear error messages, and poor onboarding guidance.

8.3 Heuristic Evaluation

The UI/UX was assessed against Nielsen's 10 Usability Heuristics, with ratings on a 1–10 scale:

← Emission mandat

Mandat minute

Mandat ordinaire

Montant (millimes)

Nom

Prénom

Numéro de téléphone

Suivant

Tunisian Post MasterCard

Figure 33: Mandate Issuance Screen

8.4 Conclusion and Recommendations

The UI/UX of the D17 application is functional and reliable for essential financial operations, particularly for users already familiar with the platform. Its simplicity, clear presentation of critical information (e.g., balance), and consistent branding contribute to a trustworthy experience aligned with financial service expectations. However, the outdated design, absence of Arabic support, navigation inefficiencies, and low user satisfaction scores indicate significant room for improvement in accessibility, modernity, and overall engagement.

To address these issues and enhance user adoption—especially in light of emerging competitors and the 2025 e-Dinar Wallet integration—the following recommendations are proposed:

- Modernize the visual design by incorporating contemporary elements such as subtle gradients, smooth animations, dark mode, and improved iconography.
- Introduce full Arabic language support and bilingual toggling to better serve Tunisia's primary linguistic context.
- Optimize navigation with grouped menu sections, a search function, and quick-access shortcuts for frequent tasks (e.g., transfers, bill payments).
- Enhance onboarding with clearer guidance, progressive forms, and tooltips to reduce drop-offs during registration.

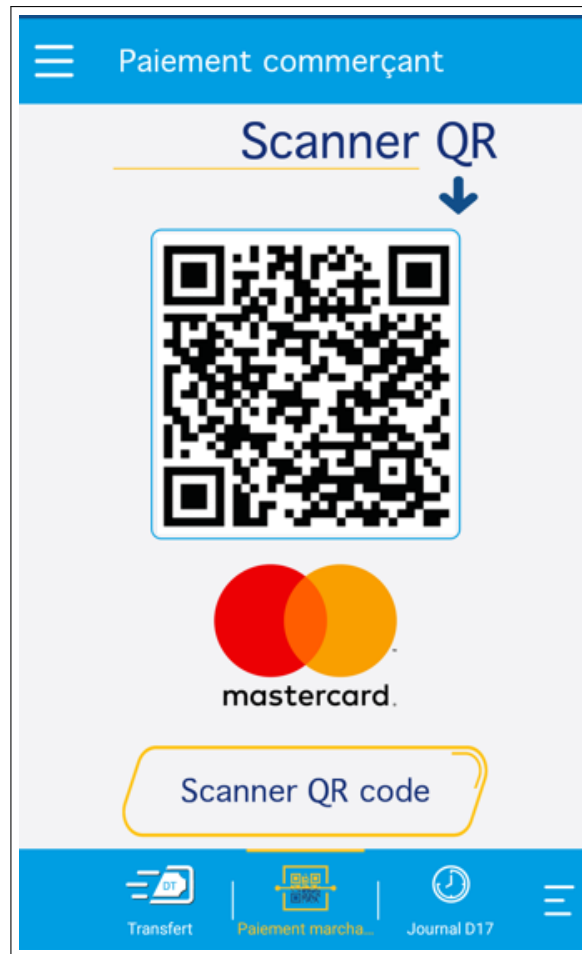


Figure 34: Merchant Payment Screen (QR Code)

- Implement biometric authentication and improve responsiveness across device sizes and orientations.
- Conduct regular user testing with diverse demographics (age, language preference, device type) to iteratively refine the experience.

Implementing these improvements would significantly elevate the application's competitiveness and user satisfaction in the Tunisian digital finance landscape.

Heuristic	Rating (1-10)	Comments
Visibility of System Status	8	Real-time balance updates are clear, but transaction confirmations often lack progress indicators.
Match Between System and Real World	7	Familiar terms used, but some jargon and French-only interface mismatch local context.
User Control and Freedom	6	Easy logout, but no undo for irreversible actions.
Consistency and Standards	7	Follows Android norms, but dated aesthetics and inconsistent icons.
Error Prevention	5	Basic validation exists, but inputs prone to errors.
Recognition Rather Than Recall	8	Icons effectively aid menu recognition.
Flexibility and Efficiency of Use	6	No shortcuts or personalization for frequent users.
Aesthetic and Minimalist Design	5	Functional but lacks modern polish and visual hierarchy.
Help Users Recognize, Diagnose, and Recover from Errors	6	Basic support available, but error messages often vague.
Help and Documentation	7	Support section exists, though depth is limited per feedback.
Overall Average	6.5	Suitable for basic tasks but needs modernization.

Table 6: Nielsen's Heuristic Evaluation for D17 App

9 Overall Analysis and Test Synthesis

Test ID	Test Description	Expected Result	Status
F-01: Login Flow			
TC_F01_001	Login with valid password	Redirect to home page	Success
TC_F01_002	Login with empty password	Display error message "Field required"	Success
TC_F01_003	Login with invalid password	Display error message "Incorrect password"	Success
TC_F01_004	Entering Arabic characters in login field	Arabic characters accepted or appropriate message displayed	Success
F-02: Registration			
TC_F02_001	Registration with empty name field	Display error message "Field required"	Success
TC_F02_002	Registration with incorrect CIN format	Display error message "Invalid format"	Success
TC_F02_003	Password masking verification	Masked characters (***) displayed	Success
F-03: Phone Top-Up			
TC_F03_001	Top-up with empty phone number	Display error message "Number required"	Success
TC_F03_002	Top-up with incorrect PIN code	Display error message "Incorrect code"	Success
TC_F03_003	Top-up with zero amount (0 TND)	Display error message "Amount must be > 0"	Success
TC_F03_004	Top-up with invalid number (7 digits)	Display error message "Invalid number"	Success
TC_F03_005	Top-up with insufficient balance	Display error message "Insufficient balance"	Success

Test ID	Test Description	Expected Result	Status
F-04: Personal Data Modification			
TC_F04.001	Password change with too short password (j 6 characters)	Display error message "Password too short"	Success
TC_F04.002	Valid password change	Display confirmation "Password updated"	Success
TC_F04.003	Profile picture change	Picture updated in profile	Success
TC_F04.004	Name change with special characters	Display error message "Unauthorized characters"	Success
F-05: Transaction Log			
TC_F05.001	Display log with no transactions	Display message "No transactions"	Success
TC_F05.002	Download log as PDF	PDF file successfully downloaded	Success
F-06: Scholarship (University Grant Payment)			
TC_F06.001	View available scholarship amount	Amount displayed correctly	Success
TC_F06.002	Scholarship withdrawal with sufficient balance	Withdrawal confirmation displayed	Success
F-07: Canteen Payment (Merchant QR Code)			
TC_F07.001	Payment with valid QR code	Payment confirmation displayed	Success
TC_F07.002	Payment with insufficient balance	Display error message "Insufficient balance"	Success
TC_F07.003	Scanning an invalid QR code	Display error message "Invalid QR code"	Success

Table 7: Test Results

10 Conclusion and Future Perspectives

10.1 Project Summary

This project successfully validated the **D17 mobile application** through a comprehensive, AI-driven quality assurance framework. By integrating the **Gemini API** for automated test generation with **Appium** for execution and **JMeter** for performance evaluation, we achieved a transition from manual verification to an efficient, scalable automated testing pipeline.

The results confirm that while the application demonstrates functional correctness and adequate security measures, the backend infrastructure encounters critical performance limitations at approximately **60 transactions per second** under heavy load. This study establishes a technical baseline for the Tunisian Post to optimize its digital services infrastructure in preparation for national peak usage periods.

10.2 Future Perspectives

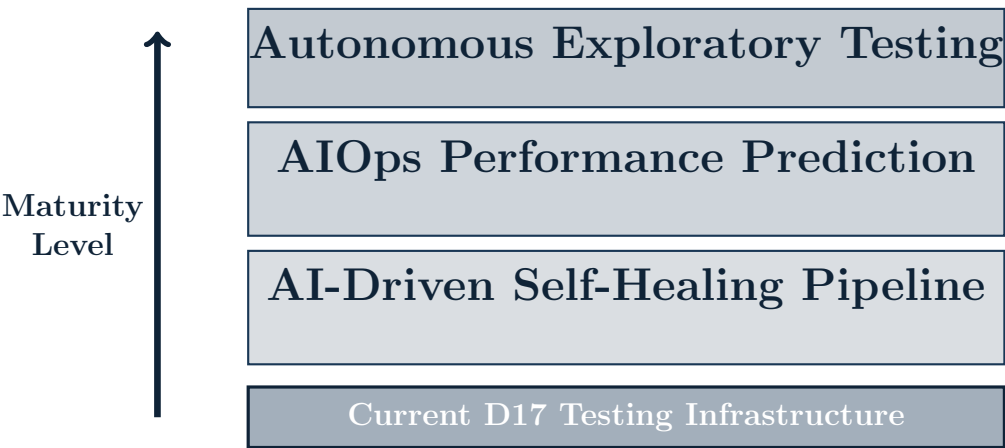


Figure 35: Evolution Roadmap for Next-Generation Testing Capabilities

The next evolution of the D17 testing infrastructure will incorporate advanced artificial intelligence capabilities:

1. **Self-Healing Test Automation:** Machine learning algorithms will automatically detect UI element changes and update test scripts without manual intervention, significantly reducing maintenance overhead.
2. **Predictive Performance Scaling:** Historical performance data will train predictive models to forecast traffic spikes and trigger preemptive infrastructure scaling, preventing service degradation during peak periods.
3. **Autonomous Exploratory Testing:** Intelligent testing agents will navigate the application like real users, discovering edge cases and defects beyond the scope of predefined test scenarios.